

PMSCS 622P

Programming Languages

Lecture 02: Variables and Data Types

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Lecture Roadmap

Today we bridge the gap between theory and code:

- 1 **Program Structure:** The skeleton of C++, Java, and Python.
- 2 **Variables & Types:** Storing data and managing memory.
- 3 **Best Practices:** Naming, Constants, and Scope.
- 4 **Implementation:** Translating step-by-step instructions into code.
- 5 **Complexity Analysis:** Introduction to Time (O) and Space efficiency.

2.1 Basic Structure (Boilerplate)

Every program has an "Entry Point" where execution begins.

C++

```
#include <iostream>
using namespace std;

// Entry Point
int main() {
    cout << "Hello";
    return 0;
}
```

Java

```
public class Main {
    // Entry Point
    public static void main(String[]
        args) {
        System.out.println("Hello");
    }
}
```

Python

```
# No boilerplate needed!
# Script executes top-to-bottom

print("Hello")
```

Note: C++ and Java require a specific function ('main') to start.

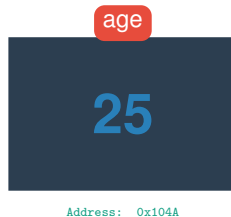
2.2 What is a Variable?

Definition

A **Variable** is a named storage location in the computer's memory (RAM) used to hold a value.

The Box Analogy:

- **Label Name** = Variable Name (*age*).
- **Contents** = Value (25).
- **Box Size** = Data Type (Integer).
- **Address** = Memory Location (*0x7fff...*).



Syntax: Declaration vs Initialization

1. Declaration: Telling the computer "I need a box named X".

```
| int score; // C++ & Java
```

2. Initialization: Putting a value inside the box.

```
| score = 100;
```

3. Combined (Best Practice):

```
| // C++ / Java  
| int score = 100;
```

Python Approach: Python does both automatically.

```
| # Python  
| score = 100
```

2.3 Identifiers (Naming Variables)

An **Identifier** is the name you give to a variable, function, or class.

The 3 Golden Rules

- 1 Can contain **Letters** (A-Z, a-z), **Digits** (0-9), and **Underscores** (_).
- 2 Must **NOT** start with a digit. (e.g., 1stPlace is illegal).
- 3 Must **NOT** be a Keyword (reserved word like `int`, `class`, `if`).

Valid	Invalid	Reason
<code>studentAge</code>	<code>student Age</code>	Contains space
<code>grade_1</code>	<code>1stGrade</code>	Starts with number
<code>_total</code>	<code>float</code>	Reserved keyword

Best Practices: Naming Styles

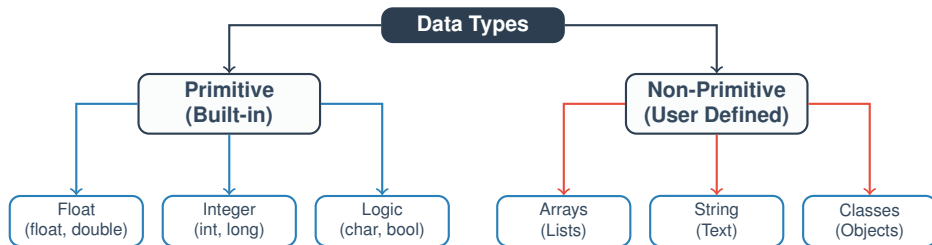
Code is read more often than it is written. Use standard styles.

camelCase (Java / C++)

- Start small, new words capitalized.
- Ex: `studentName`, `bankAccountBalance`.

snake_case (Python / C++)

- All small, separated by underscores.
- Ex: `student_name`,
`bank_account_balance`.



Primitive 1: Integers (Whole Numbers)

Used for counting items, indices, ages, etc.

Language	Type	Size (typ.)	Range
C++ / Java	int	4 bytes	-2B to +2B
C++ / Java	long	8 bytes	Huge numbers
Python	int	Dynamic	Unlimited (memory dependent)

Example

```
int population = 160000000;  
int count = -5;
```


Primitive 2: Floating Point (Decimals)

Used for measurements, currency, scientific calculations.

- **float**: Single precision (approx 7 decimal digits).
- **double**: Double precision (approx 15 decimal digits) - *Standard choice*.

Precision Warning

Computers store decimals in binary.

0.1 + 0.2 might result in 0.30000000000000004!

```
double pi = 3.14159;  
float height = 5.9f;
```

Primitive 3: Boolean (Logic)

The simplest data type. Representing logic gates.

- **Values:** true (1) or false (0).
- **Size:** 1 byte (usually).
- **Usage:** Conditions, Loops, Flags.

```
// C++ & Java  
bool isPassed = true;  
bool isRaining = false;  
  
if (isPassed) {  
    cout << "Party!";  
}
```

```
# Python (Capital T/F)  
is_passed = True
```

Primitive 4: Characters & Strings

char (Primitive)

- Single character.
- Enclosed in **Single Quotes** ('A').
- Underlying ASCII Value ('A' = 65).

String (Non-Primitive/Object)

- Sequence of characters.
- Enclosed in **Double Quotes** ("Hello").
- Can be modified, added to, sliced.



Array of Characters (C-Style String)

2.5 Static vs Dynamic Typing

This is the **major difference** between C++/Java and Python.

Statically Typed (C++, Java)

You must define the type **before** compiling.
The type cannot change.

```
int x = 10;  
x = "Hello"; // ERROR!
```

Pro: Safety, Performance.

Dynamically Typed (Python)

The type is decided at runtime. A variable can change type.

```
x = 10          # x is int  
x = "Hello"     # x is now str
```

Pro: Flexibility, Speed of coding.

Constants

Sometimes we want a variable that **cannot** change (read-only).

- **C++:** `const` keyword.
- **Java:** `final` keyword.
- **Python:** No strict constants (Convention: `ALL_CAPS`).

Why use constants?

Prevents Magic Numbers.

```
const float PI = 3.14159;  
PI = 3.0; // Compile-Time ERROR!
```

2.6 Type Casting

Converting data from one type to another.

1 Implicit (Automatic): Safe conversions (Small → Big).

int (4 bytes) → double (8 bytes)

2 Explicit (Manual): Forced conversions (Big → Small). **Risk of Data Loss!**

```
double price = 9.99;  
// Cuts off decimal part  
int cost = (int)price;  
// cost is now 9
```

9.99 (Double)



9 (Int)

0.99 Lost

Interaction: Input & Output

Variables are useless if we can't fill them with user data.

Language	Output (Print)	Input (Get)
C++	<code>cout << x;</code>	<code>cin >> x;</code>
Java	<code>System.out.println(x);</code>	Scanner class
Python	<code>print(x)</code>	<code>input()</code>

C++ Example

```
cout << "Enter Age:  ";
cin >> age;
cout << "You are " << age << " years old.";
```

2.7 Variable Scope

Scope is the region where a variable is "visible" or accessible.

Local Scope

- Inside a function/block.
- Created on entry, destroyed on exit.
- Hidden from other functions.

```
int main() {  
    int x = 10; // Local  
    if (x > 5) {  
        int y = 20; // Block Scope  
    }  
    // y is DELETED here  
    // x is still ALIVE  
}
```

Global Scope: Defined outside all functions. Accessible everywhere (Use carefully!).

Instruction to Code (C++)

Step	Instruction	C++ Code
1	Create integer quantity.	<code>int quantity;</code>
2	Assign value 5 to quantity.	<code>quantity = 5;</code>
3	Create decimal price (9.50).	<code>double price = 9.50;</code>
4	Calculate total.	<code>double total = quantity * price;</code>
5	Type cast total to integer.	<code>int exactCost = (int)total;</code>
6	Output the final cost.	<code>cout << exactCost;</code>

Instruction to Code (Java)

Step	Instruction	Java Code
1	Create integer mark1.	<code>int mark1 = 80;</code>
2	Create integer mark2.	<code>int mark2 = 90;</code>
3	Sum the marks.	<code>int sum = mark1 + mark2;</code>
4	Calculate average (divide by 2.0).	<code>double avg = sum / 2.0;</code>
5	Create a descriptive message.	<code>String msg = "Avg: " + avg;</code>
6	Print the final message.	<code>System.out.println(msg);</code>

Instruction to Code (Python)

Step	Instruction	Python Code
1	Define radius variable.	<code>radius = 5</code>
2	Define value of PI.	<code>pi = 3.1416</code>
3	Calculate area (r^2).	<code>area = pi * (radius ** 2)</code>
4	Round area to nearest whole number.	<code>result = round(area)</code>
5	Create an f-string message.	<code>msg = f"Area is {result}"</code>
6	Output the message.	<code>print(msg)</code>

Introduction to Complexity Analysis

How do we know if a program is "Good"? It's not just about getting the right answer; it's about **Efficiency**.

1. Time Complexity (Speed)

How long does the code take to run as input grows?

- **Analogy:** How long to read a book?
- 1 Page = Fast.
- 1000 Pages = Slow.

2. Space Complexity (Memory)

How much RAM (Memory) does it need?

- **Analogy:** How big is your backpack?
- 1 Notebook = Low Space.
- 50 Textbooks = High Space.

How to Compute Time Complexity?

We use **Big-O Notation** (O) to describe the "Worst Case" scenario.

Rule 1: Simple Statements are $O(1)$ Basic operations (math, assignment, checks) happen instantly.

```
int a = 10;           // 1 Step
int b = 20;           // 1 Step
int sum = a + b;      // 1 Step
// Total: Constant Time  $O(1)$ 
```

Rule 2: Loops are $O(n)$ If a loop runs N times, it takes linear time.

```
// If N is 1000, this runs 1000
// times
for(int i=0; i<N; i++) {
    cout << i;
}
// Total: Linear Time  $O(n)$ 
```

How to Compute Space Complexity?

Count how many "variables" (boxes) you are creating.

Scenario A: Fixed Variables ($O(1)$)

Regardless of input size, we only need 3 boxes.

```
int x = 5;
int y = 10;
int z = x + y;
// Uses 3 integers (12 bytes)
// always.
```

Space: Constant $O(1)$

Scenario B: Arrays/Lists ($O(n)$) We need a box for **every** item.

```
int N = 1000;
int numbers[N];
// Creates 1000 boxes!
```

Space: Linear $O(n)$

Let's Analyze This!

```
// Input: N
int total = 0;           // A

for (int i=0; i<N; i++) { // B
    total = total + i;    // C
}

cout << total;          // D
```

Step-by-Step Analysis:

- 1 **Line A:** Runs once. (1)
- 2 **Line B & C:** Runs N times. (N)
- 3 **Line D:** Runs once. (1)

Total Time: $N + 2$ operations.

Big-O: Drop constants $\rightarrow O(n)$

Space: Only 2 variables ('total', 'i').

Big-O: $O(1)$

Summary of Lecture 02

- **Variables** are named memory locations (Containers).
- **Data Types** tell the compiler how much memory to allocate.
- **Static vs Dynamic:** C++/Java (Strict) vs Python (Flexible).
- **Translation:** We learned to translate Logic Instructions directly into Syntax.
- **Complexity:**
 - **Time:** Speed ($O(1)$ vs $O(n)$).
 - **Space:** Memory Usage.

Next Lecture: [Conditionals](#).